Emaad Arsheed Hakeem 250102213

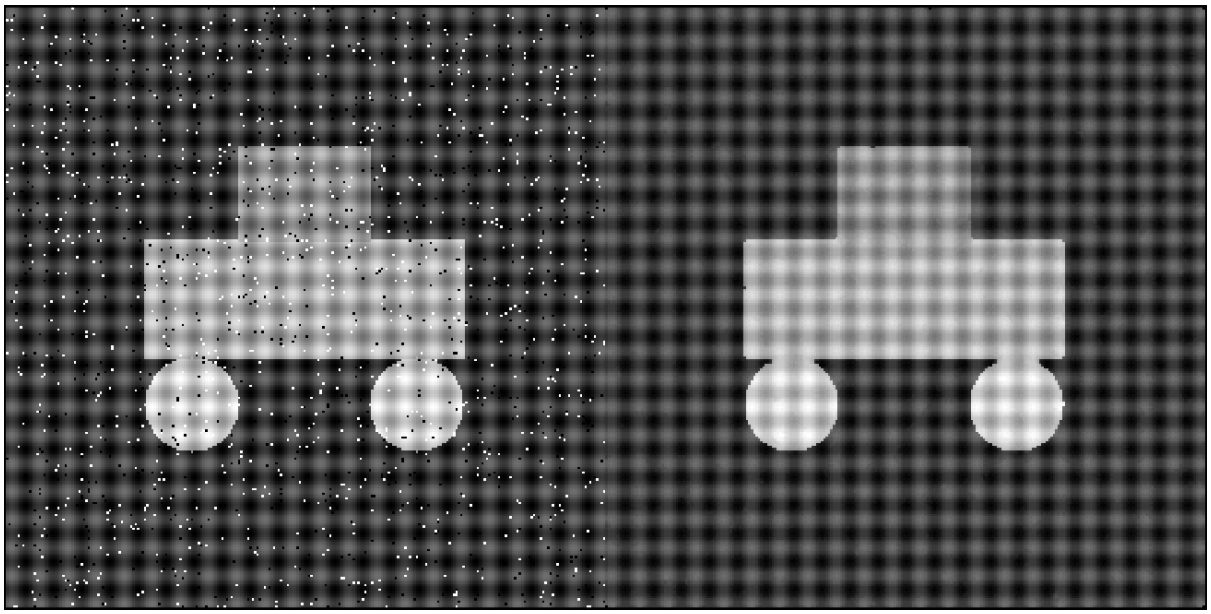
SIGNALS REPORT

STEP 1: SPATIAL MEDIAN FILTER:

Each cursed image has salt and pepper noise that needs to be removed, to do this we can use a median filter which looks at pixels in a 3x3 area and chooses the median of those and replaces the centre pixel in the 3x3 grid area with that median value. As the salt and pepper noise present in the images is made up of completely black and completely white pixels, the noise is present on the extremes when the pixels are sorted by intensity, by choosing the median pixel we can be sure that this noise will not be the median and thus will be replaced by the median.

I tested multiple different values for the size of filter but 3x3 was better because filter sizes bigger than 3x3 were giving a blurred image, and filter sizes smaller than 3x3 were not removing the salt and pepper noise completely.

Result: original → after applying special filtering



Code:

```
3
4       %using the median filter with 3x3 area to remove the salt and pepper noise
5       F1 = medfilt2(I1, [3, 3]);
6
```

STEP 2: CONVERTING IMAGE INTO FREQUENCY DOMAIN USING FFT

Code:

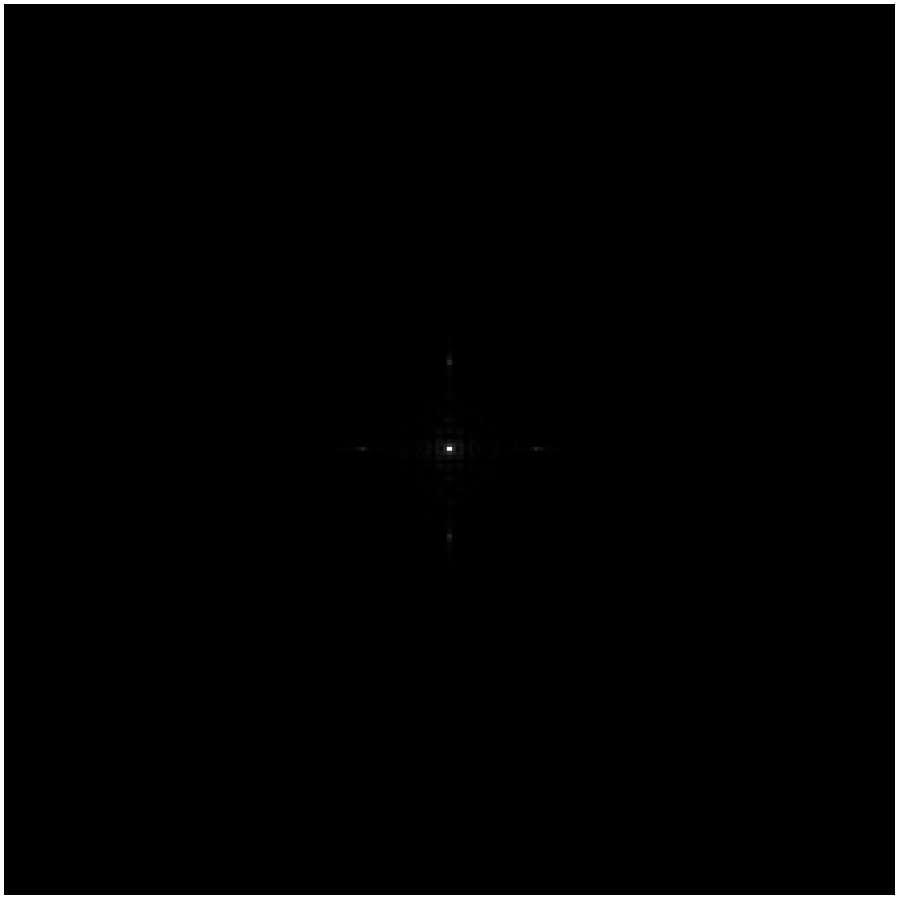
```
%using fft to convert the image into frequency medium
F1 = im2double(F1);
FFT_F1 = fft2(F1);

%shifting it so that the 0 frequency is in the middle,
%image about 0 frequency
f1shifted = fftshift(FFT_F1);

%finding absolute value because fft stores values in c
mags = abs(f1shifted);
```

To remove the repetitive patterns of dots in the image we need to first convert the image into frequency domain and remove the frequency corresponding to the noise, we can identify these frequencies as they will appear as peaks that are farther away from the origin after applying fft2(), fftshift() and abs() function . We have to apply abs() function as the fft2() function gives us a result in complex numbers. We also have to use im2double() before applying fft() to get double precision for accuracy.

After applying abs() to fftshift(). The 4 peaks shown in the image correspond to the peaks of the repeating noise. The middle peak does not represent the noise it represents the normal colour of the image. As the normal colour does not repeat it has a very low frequency close to zero. Thus the peaks close to the centre should be avoided and only the ones farther away from it should be considered as peaks due to noise

Result: Peaks in the frequency domain after applying fft(), fftshift() and abs(); the 4 peaks father away from the centre are the peaks due to the noise.

Result:



STEP 3: FINDING THE PEAKS

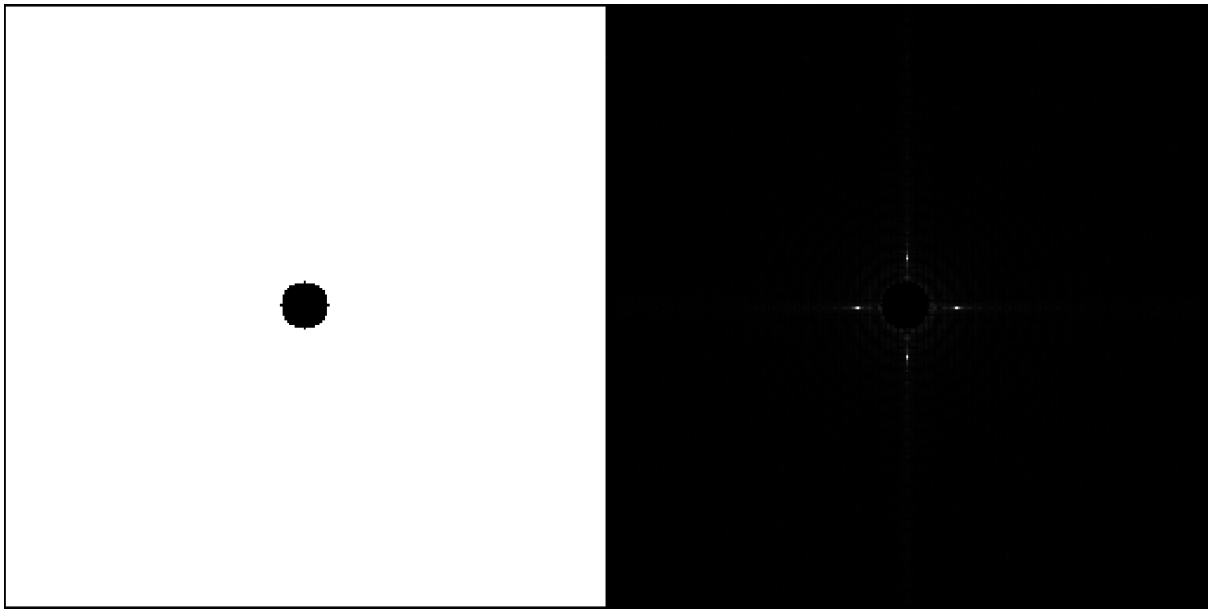
We need to find the distance of the peaks from the centre of the image but we have to ignore the peak in the centre. To do this I will first create a filter which sets all the intensities in a 10 pixel radius from the centre to 0. To remove the peak in the centre. Then use a max function to find the peaks and from there we can find the distance from that point to the centre.

First we find the size of the image using size(), then using meshgrid() we create a matrix of that size, xgrid and ygrid are the coordinates of the values or pixels in the matrix. Then we create a distances matrix which stores the values of distances of pixels from the center pixel of the image. then in only_search matrix only the values which have a distance greater than 10 from the center are 1 and the others are 0. We multiply the pixel values of the mags matrix which stores the abs values of fft image with the pixels in the onlysearch matrix. This removes the peak values in the centre of the image. Then we use max() function to find the peaks in intensity and convert it into coordinates using the ind2sub() function. Finally we find the distance of the coordinate of peak from the centre and store it in variable R.

Code:

```
%finding the dimensions of the image and making a grid of those dimensions
[columns, rows] = size(I1);
[xgrid, ygrid]  = meshgrid(1:rows, 1:columns);

%creating a grid of distances from the center
distances = sqrt((xgrid - rows/2).^2 + (ygrid - columns/2).^2);

%ignoring the extremely low frequencies because they commonly have highest peaks,
%but we only require high peaks that due to noise in the image
only_search = distances > 10;
mag_search = mags .* only_search;

%searching the image with lower frequencies removed to find the peaks
%formed due to noise
[mval, mindex] = max(mag_search(:));
[peakx, peaky] = ind2sub(size(mag_search), mindex);

%after we found the peaks we find the distance of those peaks from the
%center
R = sqrt((peakx - rows/2).^2 + (peaky - columns/2).^2);
```

Result: only_search → mags_search



STEP 4: CREATING NOTCH FILTER AND APPLYING IT

Code:

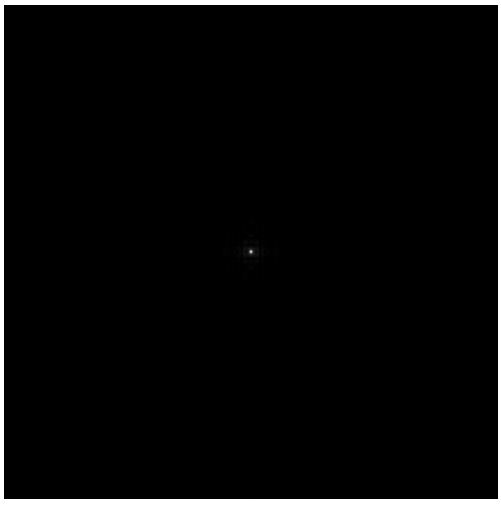
```
%after that we create filter that removes the frequencies in a small
%segment around the distance at which we found the peaks
filter = (distances > R+6) | (distances < R-6);

f_filt = f1shifted .* filter;

%we apply the inverse fft shift and fft functions to get the filtered image
filt_image_complex = ifftshift(f_filt);
filt_image = real(ifft2(filt_image_complex));
```

To remove the peaks in the frequencies we will create a notch filter around the frequency we want to remove. The notch filter will be shaped like a ring. We will then multiply the pixels of the filter with the image in the frequency domain which we get after the fft() and fftshift() functions. This will remove all the unnecessary noise in the image.

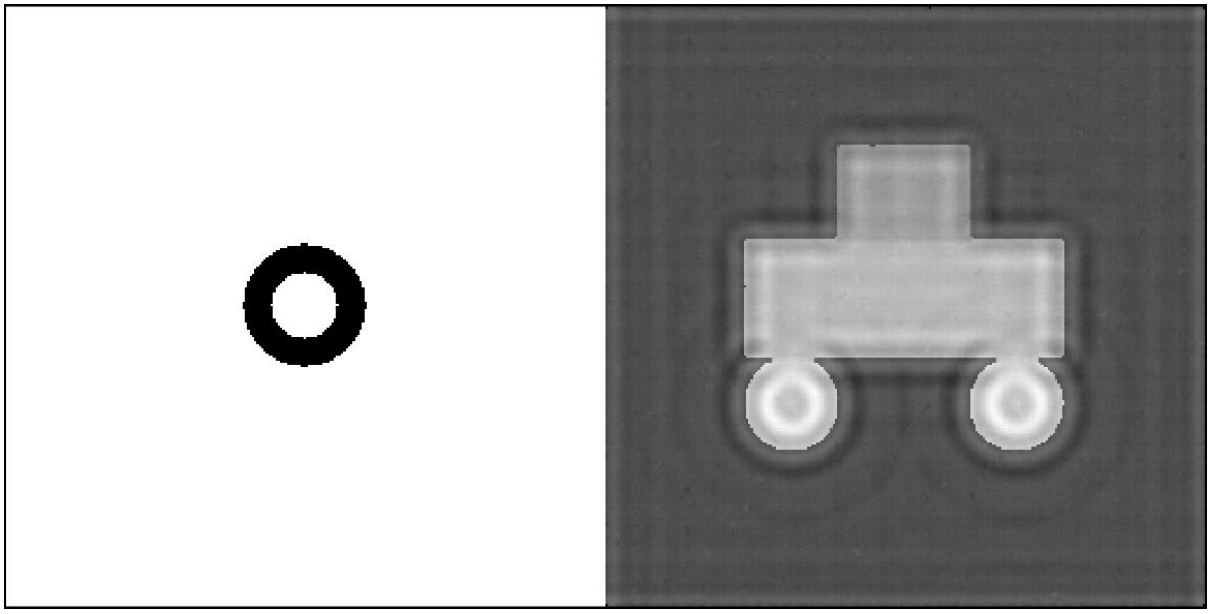
First we create a notch filter which is centered around R and has a width of 12, the width of 12 is because it is the lowest width at which I was getting clear results from each image.

We apply the filter by multiplying pixel of each filter with the pixels of f1shifted, then we apply the inverse fft function ifft() and inferse of fftshift() which is ifftshift(). Due to the floating point errors in matlab the ifftshifted() returns complex numbers with very small values for complex part, so we have to use real() function to only store the real values and ignore the complex values.

Result: f_filt (after multiplying frequency domain image with the notch filter)



Result: filter shape & filt_image (final filtered image after inverse functions are applied)



STEP 5: USING THE BLUEPRINT EFFECT

Code:

```
48
49  %we apply a blue and white blueprint filter such that the areas which have
50  %higher intensity and are lighter will be white and the areas which have a
51  %darker color and lower intensity will be blue
52  blueprint_image = (cat(3, filt_image, filt_image, 1+filt_image*0));
53
54  %showing the image
55  imshow(blueprint_image, []);
```

To create a blueprint effect we need to keep the white parts white and all the other parts we need to make blue thus, we use the cat() function which concatenates the arrays if the pixel of the filt_image has a low intensity then the blue colour of the R G B will be dominant and the final pixel will look blue, but if the intensity is high then all the colours will be high and the final pixel colour will look white.

Result: